Object-Oriented Programming

A Simple and Clear Guide

1. The Basics

What is OOP?

OOP stands for Object-Oriented Programming. It is a way of writing code where you organize everything around objects. Instead of writing a long list of instructions, you think about the things in your program and what they can do.

Think of it like the real world. A car is an object. It has properties like color and speed, and it can do things like start and stop. In OOP, you write code the same way.

OOP vs Procedural Programming

Before OOP, most code was written in a procedural style, which means you write a sequence of steps from top to bottom. It works fine for small programs, but it gets messy and hard to manage as things grow.

Procedural: You write functions and call them one after another. Everything is global and shared.

OOP: You create objects that contain both data and behavior. Each object is responsible for itself.

OOP makes your code easier to read, reuse, and maintain.

Core Concepts

Class

A class is like a blueprint or a template. It describes what an object looks like and what it can do. A class does not hold actual data by itself. It just defines the structure.

```
class Car {
    String color;
    int speed;
}
```

Object

An object is a real thing created from a class. When you create an object, you are using the blueprint to build something actual in memory.

```
Car myCar = new Car();
```

Instance

Instance is just another word for object. When you create an object from a class, you are creating an instance of that class. Every instance has its own copy of the data.

Attributes and Methods

Attributes: These are the variables inside a class. They represent the data or properties of the object, like name, age, or color.

Methods: These are the functions inside a class. They represent what the object can do, like move, print, or calculate.

Class vs Object / Attribute vs Method

Class vs Object: The class is the design. The object is the actual thing built from that design. You can build many objects from one class.

Attribute vs Method: An attribute stores data. A method does something. Think of attributes as nouns and methods as verbs.

2. Designing Classes

Class Syntax

Here is the basic structure of a class in Java:

```
class Person {
    String name;
    int age;
    void greet() {
        System.out.println("Hello, I am " + name);
    }
}
```

Constructor

A constructor is a special method that runs automatically when you create an object. Its job is to set up the object with initial values.

Default Constructor

A default constructor takes no parameters. Java adds one automatically if you do not write any constructor yourself.

```
class Person {
    String name;
    Person() {
        name = "Unknown";
    }
}
```

Parameterized Constructor

A parameterized constructor takes values when the object is created, so you can set everything up right away.

```
class Person {
    String name;
    Person(String n) {
        name = n;
    }
}
Person p = new Person("Ali");
```

The 'this' Keyword

'this' refers to the current object. You use it when the parameter name is the same as the attribute name, so Java knows which one you mean.

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}
```

Accessing Attributes and Methods

Once you have an object, you use the dot operator to access its attributes and call its methods.

```
Person p = new Person("Ali");
p.greet();
```

Destructor

A destructor is a method that runs when an object is destroyed or removed from memory. Java does not have explicit destructors. Instead, it has a garbage collector that handles memory automatically. There is a method called finalize() but it is rarely used and not recommended in modern Java.

3. Encapsulation

What is Encapsulation?

Encapsulation means hiding the internal details of an object and only exposing what is necessary. You protect the data inside a class so that it cannot be changed carelessly from outside.

Think of a TV remote. You press buttons without knowing what happens inside. The internal wiring is hidden from you. That is encapsulation.

Access Modifiers

Access modifiers control who can see or use a variable or method.

public: Anyone can access it from anywhere.

private: Only the class itself can access it. Nobody outside can touch it directly.

protected: The class and its subclasses (and classes in the same package) can access it.

Getters and Setters

When you make attributes private, you need a controlled way to read and change them. That is what getters and setters are for.

Getter: A method that returns the value of a private attribute.

Setter: A method that sets or updates the value of a private attribute.

```
class Person {
    private String name;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Direct Access vs Controlled Access

Direct access: You read or write the attribute directly. No checks, no protection.

Controlled access: You go through a getter or setter. You can add validation logic inside the setter before allowing the change.

Example: a setter can check that age is not negative before assigning it.

4. Inheritance

What is Inheritance?

Inheritance allows one class to take the properties and methods of another class. The class that gives is called the parent class (or superclass), and the class that takes is called the child class (or subclass).

This saves you from writing the same code again. The child gets everything the parent has, and can also add its own things on top.

Types of Inheritance

Single Inheritance

One child class inherits from one parent class. This is the most common type.

```
class Animal { }
class Dog extends Animal { }
```

Multilevel Inheritance

A class inherits from another class, which itself inherits from another. It is like a chain.

```
class Animal { }
class Dog extends Animal { }
class Puppy extends Dog { }
```

Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

```
class Animal { }
class Dog extends Animal { }
class Cat extends Animal { }
```

Multiple Inheritance

A class inherits from more than one parent. Java does not support this directly with classes to avoid confusion, but it can be achieved through interfaces.

Hybrid Inheritance

A combination of more than one type of inheritance. Like multilevel and hierarchical together. Java handles this through interfaces as well.

Method Overriding

When a child class has a method with the same name and parameters as a parent method, the child version replaces the parent version. This is called overriding.

```
class Animal {
    void speak() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    void speak() { System.out.println("Woof"); }
}
```

The 'super' Keyword

'super' is used inside a child class to refer to the parent class. You can use it to call the parent constructor or access the parent version of a method.

```
class Dog extends Animal {
    Dog() {
        super();
    }
}
```

Constructor Chaining

When a child object is created, the parent constructor runs first. This is called constructor chaining. Java does this automatically, but you can also call super() explicitly to pass arguments to the parent constructor.

5. Polymorphism

What is Polymorphism?

Polymorphism means one thing taking many forms. In programming, it means you can use the same method name for different behaviors depending on the situation.

It helps you write flexible and reusable code. You write one general call, and the right version of the method runs automatically.

Compile-time Polymorphism (Overloading)

This happens when you have multiple methods with the same name but different parameters. Java decides which one to call at compile time based on what you pass.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}
```

Runtime Polymorphism (Overriding)

This happens when a child class overrides a parent method, and the decision of which method to run is made at runtime based on the actual object type.

```
Animal a = new Dog();
a.speak();
```

Even though the variable type is Animal, it runs the Dog version of speak() because the actual object is a Dog.

Operator Overloading

Operator overloading means giving extra meaning to an operator like + or * depending on what it is used on. Java does not support operator overloading for custom classes. Languages like C++ and Python do support it.

6. Abstraction

What is Abstraction?

Abstraction means showing only the important parts and hiding the unnecessary details. You focus on what something does, not how it does it.

Think about driving a car. You know how to use the steering wheel and pedals, but you do not need to know how the engine works internally. That complexity is hidden from you.

Abstract Classes

An abstract class is a class that cannot be used to create objects directly. It exists as a base for other classes. It can have regular methods and abstract methods.

```
abstract class Shape {
```

```
    abstract double area();
    void describe() {
        System.out.println("I am a shape");
    }
}
```

Abstract Methods

An abstract method is a method with no body. It only has a signature. The child class is forced to provide the actual implementation.

```
class Circle extends Shape {
    double radius;
    double area() { return 3.14 * radius * radius; }
}
```

Interfaces

An interface is a step further than an abstract class. It is a contract that a class agrees to follow. All methods in an interface are abstract by default (in older Java versions), and a class can implement multiple interfaces.

```
interface Printable {
    void print();
}
class Document implements Printable {
    public void print() {
        System.out.println("Printing...");
    }
}
```

Abstract Class vs Interface

Abstract class: Can have both abstract and regular methods. Can have constructors and instance variables. A class can only extend one abstract class.

Interface: Only defines method signatures (behavior contracts). No constructors, no instance variables (only constants). A class can implement multiple interfaces.

Use an abstract class when you want to share common code between related classes. Use an interface when you want to define a contract that unrelated classes can follow.